RDFQuotient documentation

December 2, 2021

1 RDFQuotient

The RDFQuotient tool, available at the Inria GitLab repository[1], implements the RDFQuotient framework for computing quotient summaries of RDF graphs[2]. The current stable version of the software is 1.9. The latest experimental version is 2.0-SNAPSHOT. A link to download a stand-alone jar file and building-from-sources instructions (including a Docker file) can be found in the README file[3]. RDFQuotient uses PostgreSQL DBMS as a backend and it is a requirement for the setup. PostgreSQL server must be running with the specified configuration. For docker instructions, please refer to the README file.

Please read this documentation and the README carefully and thoroughly before attempting any software run to avoid data loss: the software operates on a Postgres database and thus can modify its content.

2 Command-line stand-alone application interface

The command-line interface consists of three possible operations: `load`, `summarize`, and `read`. Even though `read` operation is present in the command-line interface, it has no effect and was added for completeness. It is meant to be used programmatically. The operations follow a specific syntax that can be seen in the RDFQuotient manual in Figure 1.

Specifying `dataset.filename` argument for `--load` operation is the only way to change the input graph filename. The `--summarize` operation has two possible settings: `dataset.filename` and `database.name`. Specifying one of them is mandatory. However, if both of them are present `database.name` takes precedence. The reason for keeping the two options is that sometimes it is more convenient to use the same name for the argument, especially when we rely on the default naming of the databases created for summaries.

Example command-line execution for loading:

```
java -jar target/RDFQuotient-1.9-with-dependencies.jar --load
"dataset.filename=/data/datasets/enelshops/enelshops.nt"
```

Example command-line execution for summarization in the dry-run mode:

```
java -jar target/RDFQuotient-1.9-with-dependencies.jar --summarize
"dataset.filename=/data/datasets/enelshops/enelshops.nt" --dry-run
```

Example command-line execution for summarization:

```
java -jar target/RDFQuotient-1.9-with-dependencies.jar --summarize
"dataset.filename=/data/datasets/enelshops/enelshops.nt"
```

2.1 Configuration

The tool encapsulates hard-coded values for configuration that model all the available functionalities and scenarios. They can be overwritten by command-line arguments or by two configuration files: `loading.properties`

[1] `https://gitlab.inria.fr/cedar/RDFQuotient`

[2] `https://project.inria.fr/rdfquotient/`

[3] `https://gitlab.inria.fr/cedar/RDFQuotient/blob/master/README.md`

```
$ ./RDFQuotient.sh --help
usage: RDFQuotient 1.9

This framework is designed to work with one graph at a time.
Before using RDFQuotient make sure that the Postgres server is running.
Input RDF dataset file format is N-Triples and the file is assumed not to
contain any duplicated triples.

rdfquotient [-d] [-h | -l <[ARGS]> | -r <[ARGS]> | -s <[ARGS]> | -v]  [-lp
       <filename> | -sp <filename>]
-d,--dry-run                                    - print the configuration used
                                                for a run
-h,--help                                       - print help message
-l,--load <[ARGS]>                              - load the dataset into database
                                                [ARGS] must specify a value for
                                                dataset.filename
                                                CAUTION: database is dropped by
                                                default
-lp,--use-loading-properties <filename>         - set loading properties
                                                filename
-r,--read <[ARGS]>                              - read summary from database
                                                [ARGS] must specify a value for
                                                database.name
-s,--summarize <[ARGS]>                         - summarize the dataset
                                                [ARGS] must specify a value for
                                                dataset.filename or
                                                database.name
-sp,--use-summarization-properties <filename>   - set summarization properties
                                                filename
-v,--version                                    - print the version of
                                                RDFQuotient

[ARGS] is a comma-separated list of assignments of form key=value, where key is a
configuration property from the list of loading or summarization configuration
properties.
```

Figure 1: Command-line interface of RDFQuotient

and `summarization.properties`. The command-line arguments and configuration files are optional. The configuration files serve the purpose of templates, they are filled in with the default, hard-coded values.

To determine the configuration for a run of the tool for either loading or summarization properties files we apply Algorithm 1. Hard-coded values have the lowest priority. Values present in the configuration files have medium priority. Values passed in the arguments have the highest priority.

2.2 Configuration files

The following two lists contain all the functionalities and scenarios in terms of the configuration parameters of the execution.

2.2.1 Loading properties

The list of loading configuration properties with default values assigned:

Algorithm 1 Setting up of the configuration

1: **procedure** setUpConfiguration
2: **if** `args["properties_filename"]` $\neq$ `null` **then**
3: `properties_filename` $\leftarrow$ `args["properties_filename"]`
4: **if** `properties_filename` file exists **then**
5: overwrite the default, hard-coded values for `properties` with the values in `properties_filename` file
6: **for each** `setting` $\in$ `args.values` **do**
7: overwrite the value of `setting` in `properties` with the value specified by `args[setting]`

1. `dataset.filename = test.nt`
   Caution: RDFQuotient assumes the N-Triples format for the dataset file and no duplicates present.
2. `database.host = localhost`
3. `database.port = 5432`
4. `database.user = postgres`
5. `database.password = postgres`
6. `database.name =`
   Blank by default.
   If left blank, the database name will be derived from the dataset filename. Otherwise, the specified name will be used.
7. `database.drop_existing_db = true`
   Valid options: `true`, `false`.
   Caution: if the `database.name` exists, it is dropped by default before loading.
8. `database.storage_layout = TRIPLES_TABLE`
   Valid options: `TRIPLES_TABLE`, `TABLE_PER_ROLE_AND_CONCEPT`.
9. `database.triples_table_name = triples`
10. `database.encoded_triples_table_name = encoded_triples`
11. `database.encoded_saturated_triples_table_name = encoded_saturated_triples`
12. `database.dictionary_table_name = dictionary`
13. `dictionary.fetch_size = 1000`
14. `saturation.type = NONE`
    Which saturation algorithm to use while loading the graph. Valid options: `NONE`, `CONSTRAINT_SAT`, `ASSERTION_SAT`, `RDFS_SAT`.
15. `saturation.enable = false` *(deprecated)*
    Valid options: `true`, `false`. The `true` value corresponds to the `ASSERTION_SAT` algorithm. The `saturation.type` option has a higher priority.
16. `saturation.batch_size = 1000`
17. `database.cluster_indexes = true`
    Whether to add clustered indexes to the tables created by OntoSQL (enabling this is recommended for better performance).
18. `database.deterministic_ordering = false`
    Whether to sort dictionary entries before assigning encodings and also `encode_triples` and `encoded_saturated_triples` tables. This is used for tests, and should not be used for large graphs.

19. `statistics.export_to_csv_file = true`
    Valid options: `true`, `false`.

20. `configuration.export_to_disk = false`
    Whether to export `loading.properties` used in a run after it's finished.
    Valid options: `true`, `false`.

2.2.2 Summarization properties

The list of summarization configuration properties with default values assigned:

1. `dataset.filename = test.nt`

2. `database.host = localhost`

3. `database.port = 5432`

4. `database.user = postgres`

5. `database.password = postgres`

6. `database.name =`
    Blank by default.
    If left blank, the database name will be derived from the dataset filename. Otherwise, the specified name will be used.

7. `database.triples_table_name = triples`

8. `database.encoded_triples_table_name = encoded_triples`

9. `database.dictionary_table_name = dictionary`

10. `database.encoded_saturated_triples_table_name = encoded_saturated_triples`

11. `database.deterministic_ordering = false`
    Whether to sort the `encoded_triples` and `encoded_saturated_triples` tables. This is used for tests, and should not be used for large graphs.

12. `summary.summarize_saturated_graph = false`
    Raises an error in case of absence of saturated triples table.
    Valid options: `true`, `false`.

13. `summary.type = strong`
    Valid options: `typed`, `2pinputoutput`, `2ponefw`, `2ponefb`, `2pstrong`, `2pweak`, `2pweakunionfind` (an implementation variant of `2pweak` but less efficient), `2ptypedstrong`, `2ptypedweak`, `strong`, `weak`, `typedstrong`, `typedweak`.

14. `summary.omit_generic_properties_from_cliques = true`
    Valid options: `true`, `false`.

15. `summary.generic_properties = <http://www.w3.org/2000/01/rdf-schema#label>,`
    `<http://www.w3.org/2000/01/rdf-schema#comment>,`
    `<http://www.w3.org/1999/02/22-rdf-syntax-ns#value>,`
    `<http://www.w3.org/2002/07/owl#sameAs>,`
    `<http://rq.org/nodeSupport>,`
    `<http://rq.org/edgeSupport>,`
    `<http://rq.org/edge>,`
    `<http://rq.org/reifiedEdgeSubject>,`
    `<http://rq.org/reifiedEdgeProperty>,`
    `<http://rq.org/reifiedEdgeObject>`
    List of comma-separated values of URI within angle brackets.

16. `summary.replace_type_with_most_general_type = false`
    Raises an error in the case of the `W` and `S` summaries as they do not follow the "type-then-data" approach.
    Valid options: `true`, `false`.

17. `summary.default_type_property_URI`
    The URI specifying the default *rdf:type* property URI. This property is output in the summary type triples (regardless whether the original graph triple used this URI or not). This property may be a user-defined URI and the actual set of properties to be considered as type properties can be specified in the variant properties. Constraint: the default property cannot be an empty string.

18. `summary.variant_type_property_URIs`
    A list of variant type property URIs. These properties are treated as if they were semantically equivalent to *rdf:type*. Together the default and the variant type properties form all type properties. All the type properties are used to check whether the edge label in an RDF triple represents *semantically* a type annotation. For example, setting the default to *rdf:type* (`http://www.w3.org/1999/02/22-rdf-syntax-ns#type`) and the variant list to *instanceOf* (`http://www.wikidata.org/prop/direct/P31`) (a list of size 1), both the *rdf:type* and *instanceOf* triples are considered to mean the type, semantically. However, in the output, the type properties will only be assigned the default *rdf:type* label (even if in the original graph they were labeled with the variant *instanceOf* label).

19. `summary.consistency_checks = false`
    This may make summarization significantly slower; set it to `true` only for debugging.
    Valid options: `true`, `false`.

20. `summary.export_to_database = true`
    Valid options: `true`, `false`.

21. `summary.export_representation_function_and_node_statistics_to_database = true`
    Whether to export the representation function to NT file while exporting to database.

22. `summary.export_representation_function_to_nt_filename = representation_function.nt`
    The filename of an NT file to export the representation function to while exporting to database. The file is stored under the subdirectory specified by in the `summary.nt_file_prefix` option. Disabled if given the empty string `""`.

23. `summary.export_node_statistics_to_nt_filename = node_statistics.nt`
    The filename of an NT file to export the node statistics to while exporting to database. The file is stored under the subdirectory specified by in the `summary.nt_file_prefix` option. Disabled if given the empty string `""`.

24. `summary.export_edge_statistics_to_nt_filename = edge_statistics.nt`
    The filename of an NT file to export the edge statistics to while exporting to database. The file is stored under the subdirectory specified by in the `summary.nt_file_prefix` option. Disabled if given the empty string `""`.

25. `summary.add_representation_counts_in_nt_and_dot_files = true`
    Whether to add representation count statistics in the NT and DOT summary files.
    Valid options: `true`, `false`.

26. `summary.export_to_nt_file = true`
    Whether to export the summary to an NT file.
    Valid options: `true`, `false`.

27. `summary.nt_file_prefix = summariesNT/`
    Prefix added to summary NT file, usually to dispatch it into a separate directory.

28. `summary.step_by_step = false`
    Whether to execute step-by-step summarization. If true, then it is recommended to set `drawing.step_by_step = always`.

29. `drawing.dot_installation = /usr/local/bin/dot`
    Path to the DOT executable. If this is not found, it will not cause an error, only the drawing will not work.

30. `drawing.dot_file_prefix = summariesDOT/`
    Prefix added to the summary DOT file, usually to dispatch it into a separate directory.

31. `drawing.png_file_prefix = summariesPNG/`
    Prefix added to the summary PNG file, usually to dispatch it into a separate directory.

32. `drawing.remove_dot_file = false`
    Whether to remove the DOT file in case of successful execution of DOT.

33. `drawing.draw_input_graph = false`
    Whether to draw input RDF graph.
    Valid options: `true`, `false`.

34. `drawing.step_by_step = false`
    Valid options: `false`, `always`, `when_changes`.
    Whether to draw step-by-step drawings to track the incremental summary construction. The `when_changes` option enables drawings only once a summary changes upon a triple addition. The `always` option draws upon each addition.

35. `drawing.style = split_and_fold_leaves`
    Valid options: `plain`, `split_leaves`, `split_and_fold_leaves`.
    The `plain` option draws a simple graph, whereas `split_leaves` divides sink graph nodes, enabling a leaf-to-parent nodes inlining applied further if the `split_and_fold_leaves` option is used. If an invalid option is specified, the drawing will be suppressed.

36. `drawing.color_scheme = diverse`
    Valid options: `diverse`, `bw`, `ivory`, `light`.

37. `drawing.title = true`
    Valid options: `true`, `false`.

38. `drawing.max_node_label_length = 20`

39. `drawing.max_types_drawn_per_namespace = 5`

40. `drawing.summary_node_URI_prefix = http://rq.org/`

41. `drawing.summary_node_support_URI_prefix = http://rq.org/nodeSupport`

42. `drawing.summary_edge_support_URI_prefix = http://rq.org/edgeSupport`

43. `drawing.reified_summary_edge_URI_prefix = http://rq.org/edge`

44. `drawing.reified_edge_subject_URI_prefix = http://rq.org/reifiedEdgeSubject`

45. `drawing.reified_edge_property_URI_prefix = http://rq.org/reifiedEdgeProperty`

46. `drawing.reified_edge_object_URI_prefix = http://rq.org/reifiedEdgeObject`

47. `statistics.export_to_csv_file = true`
    Valid options: `true`, `false`.

48. `configuration.export_to_disk = false`
    Valid options: `true`, `false`.
    Whether to export `summarization.properties` used in a run after it's finished.

3 Programmatic Java interface

The same set of options as for the stand-alone interface discussed above, is also available in the programmatic Java user interface through the `fr.inria.cedar.RDFQuotient.controller.Interface` class.

Here, instead of passing command-line arguments to the program, we pass a Java `Properties` object to one of the dedicated methods. We can explicitly set values of some configuration properties using other dedicated methods. We can also access the Java `Connection` object (the JDBC handler) and not close it if we don't want to. It makes this implementation stateful as it can potentially use a connection opened outside of this class and can return it to be used later. On the contrary, the command-line implementation is stateless.

See the detailed explanation of methods in Interface class below:

1. `public static Connection getOrEstablishNewDatabaseConnection(Properties properties)`
   If the database connection is established the method returns it, otherwise the method establishes a new connection using the configuration specified in `properties`.

2. `public static Connection getDatabaseConnection()`
   Returns a database connection. The method may return `null` if the database connection has not been established yet or has been already closed.

3. `public static void closeDatabaseConnection()`
   Closes database connection.

4. `public static void forceCloseDatabaseConnection(Properties properties)`
   This is an emergency method if a library/driver misbehaves and doesn't close the connection. The method should not be used otherwise.

5. `public static String load(String configurationFilename, Properties properties, boolean closeConnection)`
   If both `configurationFilename` and `properties` are `null` or point to a file/object that does not contain valid configuration properties, the default values will be used.
   Otherwise, the following procedure applies:

   (a) Take the default values;

   (b) Overwrite them with the values from the `configurationFilename` file;

   (c) Overwrite them with the values from the `properties` object.

   The `closeConnection` parameter controls whether to close the connection to the database after the call to this method.
   Examples:

   (a)
   ```
   Properties myProperties = new Properties;
   myProperties.put("dataset.filename", "datasets/my_dataset.nt");
   Interface.load(LoadingProperties.getDefaultLoadingPropertiesFilename(),
   myProperties, true);
   ```

   (b)
   ```
   Properties myProperties = LoadingProperties.getDefaultProperties();
   myProperties.put("dataset.filename", "datasets/my_dataset.nt");
   Interface.load(null, myProperties, true);
   ```

   (c)
   ```
   Properties myProperties = LoadingProperties.getDefaultProperties();
   myProperties.put("dataset.filename", "datasets/my_dataset.nt");
   Interface.load("conf/my_config_file.properties", myProperties, true);
   ```

   Returns a name of the database.

6. `public static HashMap<String, String> summarize(String configurationFilename, Properties properties, boolean closeConnection)`
   See the `load` method comment: in the examples, we use the `SummarizationProperties` class instead

of `LoadingProperties`.
Returns a map with keys: `"databaseName"`, `"NTFilename"` and `"DOTFilename"`. If the corresponding name is not present, the value is set to `null`.

7. `public static Summary read(String configurationFilename, Properties properties, boolean closeConnection)`
   See the `load` method comment.
   Returns Summary object.

8. `public static HashMap<String, String> loadAndSummarize(Properties loadingProperties, Properties summarizationProperties)`
   For convenience, this method combines the functionalities of `load` and `summarize` in one. By default, the database connection is closed.

9. `public static HashMap<String, String> loadAndSummarizeThroughShortcut(Properties loadingProperties, Properties summarizationProperties)`
   For convenience, this method combines the functionalities of `load` and `summarize` in one as well, this time through the shortcut procedure, instead. By default, the database connection is closed.

4 Tests

Tests are stored in the `src/test` directory. Resources are stored in the `src/test/resources` directory that contains all the input test files for our automatic correctness tests (not to be confused with `consistencyChecks` which are tests that can be performed during the execution of the program depending on the configuration with solely debugging purposes). The tests are checked once the `mvn install` command is run. They can be skipped by adding a flag `-DskipTests`, i.e., running the `mvn clean install -DskipTests` command.

5 Scripts

Project tree contains the `scripts` directory with useful bash scripts for computing the summaries with different configurations specified. These scripts account for memory settings of the Java Virtual Machine, as well as they can realize shortcut computations using a stand-alone interface by composing a pipeline of commands.